

From Neurons to Frontier Architecture

One unbroken derivation: the single neuron → backpropagation → reinforcement learning → the Transformer → GPT-2 → RMSNorm/RoPE/GQA/SwiGLU → Mixture-of-Experts & Multi-Head Latent Attention. Every step motivated by the specific problem it solved, every mechanism solved by hand with real numbers.

Companion to: *LLM Internals Lab* (01–06), *Qwen2.5 Architecture* (this document assumes and extends both) | Sister document: *The Architecture of Intelligence, 2nd Edition* — that book surveys 27 papers/concepts across the whole stack (data, post-training, serving, agents, eval); this document does one thing only — derives the model architecture itself, start to finish, nothing skipped. | All worked examples computed live by the build scripts in this folder, not hand-transcribed.

How to read this

This document is a **chain**, not a reference. Each part exists because the part before it hit a wall — a specific, nameable limitation — and each part's ending is the next part's motivation. The **amber boxes** mark every joint in that chain explicitly: what broke, what fixed it, what broke next. If you only read the amber boxes start to finish, you have the whole argument of the book in nine sentences.

Blue **prompt boxes** appear wherever an architectural change has an observable fingerprint — an actual prompt you can run against a real model to see the consequence of that design choice, not just read about it.

Contents

Part I — Neural Network Foundations 1. The single neuron 2. From one neuron to a network 3. Measuring wrongness: loss functions 4. Gradient descent 5. Backpropagation, solved by hand	Part II — Reinforcement Learning Foundations 6. Agent, environment, reward, policy 7. Policy gradients: a worked bandit 8. Why LLMs need RL — the bridge to RLHF/RLVR
Part III — The Transformer, From Scratch 9. Why recurrence wasn't enough 10. Self-attention with learned projections 11. Multi-head attention 12. Positional encoding, the original sinusoidal form 13. The encoder block, assembled 14. Decoder-only: why GPT dropped the encoder	Part IV — GPT-2: Checkpoint 15. GPT-2, recapped against what you already built
Part V — What Changed After GPT-2 16. RMSNorm 17. RoPE 18. Grouped-Query Attention 19. SwiGLU	Part VI — 2026 State of the Art 20. Mixture-of-Experts 21. Multi-Head Latent Attention 22. The landscape beyond attention
Part VII — Closing 23. The whole chain, one page 24. Prompt library, collected 25. Glossary & files	

The nine-sentence version

1. A single neuron is a weighted vote that can only draw a straight line; stacking neurons in layers with a nonlinearity between them lets the network draw curves — but only if you can compute *which direction* to nudge millions of weights, which is what backpropagation is.
2. Backpropagation tells you how to improve a network once you know what "wrong" means for a single labeled example — but language models must eventually be judged on things with no single correct label (helpfulness, harmlessness), which needs reinforcement learning: optimizing against a reward signal instead of a fixed target.
3. Feeding text through a network one token at a time (RNNs) makes training sequential and slow, and long- range dependencies fade — attention replaces sequential recurrence with a single parallel operation over all positions at once.
4. The original Transformer (2017) used attention inside a heavy encoder-decoder built for translation; GPT-2 (2019) showed a decoder-only stack, trained only to predict the next token, generalizes to nearly any task — the architecture you already built by hand.
5. GPT-2's LayerNorm, learned position table, one-K/V-per-head attention, and GELU MLP each carried a cost that only became visible at larger scale and longer context — RMSNorm, RoPE, GQA, and SwiGLU are each a direct fix for one specific cost (Part V — and the subject of the companion *Qwen2.5 Architecture* document).
6. Even GQA's shared-KV-head trick eventually hits a memory wall at frontier context lengths and model sizes — Multi-Head Latent Attention compresses the cache further still, via a fundamentally different mechanism (low-rank projection, not head-sharing).
7. Making a dense model bigger makes every token pay for the whole model's compute regardless of how hard that token is — Mixture-of-Experts routes each token to a handful of specialized sub-networks instead, decoupling total knowledge (parameters) from per-token cost (activated compute).
8. None of RMSNorm, RoPE, GQA, SwiGLU, MLA or MoE change what the model is trained to do — next-token prediction, refined by RL — they only change how efficiently that training and inference can happen at scale.
9. Every one of these nine sentences is a single row in the "what moved, what's the new constraint" table in Part VII — read that table first if you want the map before the territory.

PART I

NEURAL NETWORK FOUNDATIONS

The single neuron, stacked into layers, trained by gradient descent and backpropagation.
Everything downstream — attention, RoPE, MoE — is built out of exactly these five ideas, repeated.

1. The single neuron

What it is: a neuron takes several numbers in, multiplies each by a learned weight, adds a learned bias, sums the result, and passes that sum through a nonlinear function. That's the entire unit. Everything in this document — attention heads, MoE experts, the whole 671-billion-parameter DeepSeek-V3 — is this same operation, repeated and wired together in different patterns.

```
z = w·x + b      (weighted sum plus bias — a linear operation)
a = σ(z)         (activation function — a nonlinear "squash," why the network isn't just linear algebra)

σ(z) = 1 / (1 + e-z)      sigmoid: squashes any real number into (0, 1) — useful for "yes/no" outputs
```

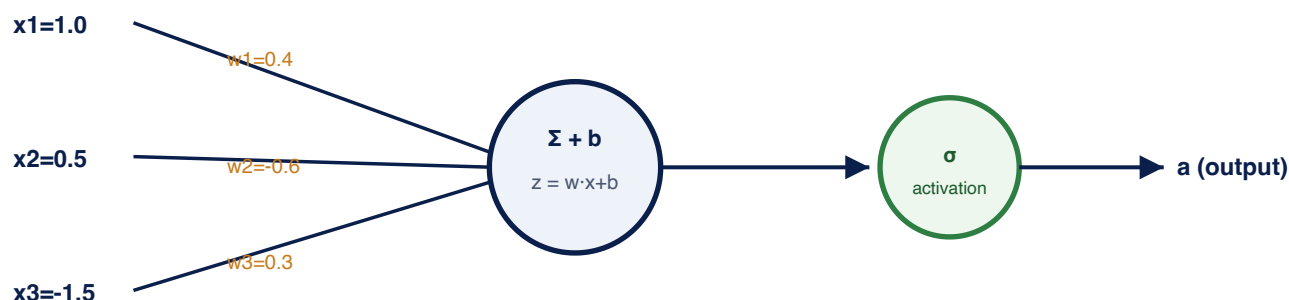


Figure 1. One neuron: a weighted sum of its inputs plus a bias, squashed by a nonlinear activation function. This is the entire computational unit — every network in this document is just many of these, wired together.

Worked example — one neuron, three inputs

```
x = [ 1.   0.5 -1.5]   (inputs)
w = [ 0.4 -0.6  0.3]   (learned weights)
b = 0.1   (learned bias)
```

Step 1 — weighted sum:

```
w·x = (0.4)*(1.0) + (-0.6)*(0.5) + (0.3)*(-1.5)
     = 0.40 + -0.30 + -0.45 = -0.35
```

Step 2 — add bias:

```
z = -0.35 + 0.1 = -0.25
```

Step 3 — squash with sigmoid:

```
a = 1 / (1 + e-(-0.25)) = 0.4378
```

Answer: a = 0.4378

Why nonlinearity matters: without σ , stacking neurons would just be stacking linear operations — and a linear function of a linear function is still linear. No amount of depth would let the network represent a curve. The nonlinearity is what makes "deep" mean anything.

2. From one neuron to a network

A **layer** is several neurons run in parallel on the same input, each with its own weights — written as one matrix multiply instead of a loop. Stack layers so one layer's output feeds the next layer's input, and you have a network. The middle layer(s) are called **hidden** layers because nothing outside the network ever looks at their values directly.

```
Layer as matrix multiply (n_in inputs, n_out neurons):
z = W·x + b      W has shape (n_out, n_in), b has shape (n_out,)
h = activation(z)

Two-layer network:
h = ReLU(W1·x + b1)      ReLU(z) = max(0, z) — the standard hidden-layer nonlinearity
y = sigmoid(W2·h + b2)
```

Worked example — forward pass through a 2-input → 2-hidden → 1-output network

```
x = [1. 2.]
W1 = [[0.5, -0.3], [0.2, 0.8]]    b1 = [ 0.15 -0.1 ]
W2 = [ 0.6 -0.4]                  b2 = 0.05
```

```
Step 1 - hidden pre-activation: h_pre = W1 @ x + b1
h_pre[0] = 0.5*1.0 + -0.3*2.0 + 0.15 = 0.05
h_pre[1] = 0.2*1.0 + 0.8*2.0 + -0.1 = 1.70
h_pre = [0.05 1.7 ]
```

```
Step 2 - ReLU: h = max(0, h_pre)
h = [0.05 1.7 ]
```

```
Step 3 - output pre-activation: y_pre = W2 @ h + b2
y_pre = 0.6*0.05 + -0.4*1.70 + 0.05 = -0.6000
```

```
Step 4 - sigmoid: y = sigmoid(y_pre)
y = 0.3543
```

Answer: $y = 0.3543$ (the network's prediction, before it has learned anything)

3. Measuring wrongness: loss functions

A loss function turns "how wrong was the prediction" into a single number the network can be pushed to reduce. For a yes/no target ($t=0$ or 1), **binary cross-entropy** is the standard choice — it penalizes confident wrong answers far more harshly than a squared-error loss would.

$$\text{BCE}(y, t) = -[t \cdot \log(y) + (1-t) \cdot \log(1-y)]$$

Worked example — loss of the prediction above, against target $t=1$

```
y = 0.3543, t = 1.0
BCE = -[1 * log(0.3543) + 0 * log(1-0.3543)] = -log(0.3543) = 1.0375
```

Answer: loss = 1.0375

If the network had instead predicted $y=0.99$ (confidently and correctly), the loss would be $-\log(0.99) = 0.0101$ — near zero. If it had predicted $y=0.01$ (confidently and *wrong*), the loss would be $-\log(0.01) = 4.6$ — the penalty grows without bound as confidence in the wrong answer increases. This is exactly the same cross-entropy loss your `03_train_tiny_gpt.py` minimizes at every training step, just extended from one yes/no output to one probability per vocabulary token.

4. Gradient descent

If you know which direction increases the loss, moving every weight a small step in the *opposite* direction decreases it. That derivative — "how loss changes as this one weight changes" — is the gradient. Gradient descent is nothing more than: compute the gradient, take a small step against it, repeat.

$$w_{\text{new}} = w_{\text{old}} - \eta \cdot (d\text{Loss}/dw) \quad \eta \text{ (eta) = learning rate: how big a step to take}$$

Too large an η overshoots and diverges; too small wastes compute crawling toward the minimum. This one line, applied to every one of a model's parameters, is the entire training loop — `torch.optim.AdamW` in `03_train_tiny_gpt.py` is a more sophisticated version of exactly this update rule.

5. Backpropagation, solved by hand

Gradient descent needs $d\text{Loss}/dw$ for every weight in the network, not just the last layer's. Backpropagation computes all of them in one backward sweep, by repeatedly applying the chain rule: the gradient of the loss with respect to an early-layer weight is the product of local derivatives strung together, layer by layer, from the loss back to that weight.

forward pass (compute the prediction) →



← backward pass (chain rule, layer by layer)

$$dL/dW1 = (dL/dh \cdot \text{ReLU}') \otimes x \quad dL/dW2 = (y-t) \cdot h, \quad dL/dh = (y-t) \cdot W2 \quad dL/dy_{\text{pre}} = y-t$$

The chain rule, applied mechanically, layer by layer, back to front

Every gradient the loss needs is a product of local derivatives strung together — exactly what PyTorch's `.backward()` automates, and what Section 5's worked example does by hand.

Figure 2. Forward pass computes a prediction and a loss; backward pass reuses the exact same computational graph in reverse to compute how every single weight should move.

Worked example — full backward pass through the network from Section 2, then one gradient step

Forward pass recap: $h = [0.05 \ 1.7]$, $y = 0.3543$, $t = 1.0$, $\text{loss} = 1.0375$

Step 1 — gradient at the output (sigmoid+BCE combine into this clean form):

$$dL/dy_{\text{pre}} = y - t = 0.3543 - 1.0 = -0.6457$$

Step 2 — gradient onto $W2$, $b2$ (chain rule: $dL/dW2 = dL/dy_{\text{pre}} \cdot h$):

$$dL/dW2 = -0.6457 \cdot [0.05 \ 1.7] = [-0.0323 \ -1.0976]$$
$$dL/db2 = -0.6457$$

Step 3 — gradient flowing backward into h (chain rule: $dL/dh = dL/dy_{\text{pre}} \cdot W2$):

$$dL/dh = -0.6457 \cdot [0.6 \ -0.4] = [-0.3874 \ 0.2583]$$

Step 4 — through the ReLU (its local derivative is 1 where $h_{\text{pre}} > 0$, else 0; both units were > 0 here):

$$dL/dh_{\text{pre}} = dL/dh \cdot \text{ReLU}'(h_{\text{pre}}) = [-0.3874 \ 0.2583] \cdot [1. \ 1.] = [-0.3874 \ 0.2583]$$

Step 5 — gradient onto $W1$, $b1$ (chain rule: $dL/dW1 = \text{outer}(dL/dh_{\text{pre}}, x)$):

$$dL/dW1 = \text{outer}([-0.3874 \ 0.2583], [1. \ 2.]) =$$
$$\begin{bmatrix} -0.3874 & -0.7748 \\ 0.2583 & 0.5165 \end{bmatrix}$$
$$dL/db1 = [-0.3874 \ 0.2583]$$

Step 6 — gradient descent update, learning rate 0.1:

$$W2_{\text{new}} = [0.6 \ -0.4] - 0.1 \cdot [-0.0323 \ -1.0976] = [0.6032 \ -0.2902]$$
$$b2_{\text{new}} = 0.05 - 0.1 \cdot -0.6457 = 0.11457$$
$$W1_{\text{new}} =$$
$$\begin{bmatrix} 0.5387 & -0.2225 \\ 0.1742 & 0.7483 \end{bmatrix}$$
$$b1_{\text{new}} = [0.1887 \ -0.1258]$$

Step 7 — verify it worked: forward pass again with the NEW weights on the SAME input:

$$\text{new prediction } y = 0.4592 \quad (\text{was } 0.3543)$$
$$\text{new loss} = 0.7782 \quad (\text{was } 1.0375)$$

Answer: loss dropped from 1.0375 to 0.7782 after exactly one gradient step

This is the entire content of what `loss.backward() + opt.step()` do in `03_train_tiny_gpt.py` — computed automatically, for every one of the model's 27,264 parameters, at every one of 2,000 training steps, by exactly this chain-rule bookkeeping. Nothing about backpropagation changes for a bigger network — more layers just means more links in the same chain.

CHAIN → Backprop tells you how to improve a prediction against a **KNOWN** correct answer. But "was this response helpful and harmless" has no single correct label to backpropagate against — that needs a different training paradigm entirely: reinforcement learning. Part II.

PART II

REINFORCEMENT LEARNING FOUNDATIONS

Why "predict the correct next token" eventually stops being the right training signal, and what replaces it.

6. Agent, environment, reward, policy

Everything in Part I assumed a **known correct answer** for every example — that's what made backpropagation possible: compute exactly how wrong the output was, and in exactly which direction. Reinforcement learning (RL) is for problems where there is no single correct answer, only a way to score how good an outcome was after the fact. Four pieces of vocabulary, defined precisely because they get reused, renamed, directly for LLM training:

- **Agent** — the thing making decisions. For an LLM being post-trained with RL, the agent *is* the model.
- **Environment** — everything the agent's actions affect and that affects it back. For an LLM, the "environment" is as simple as: the conversation so far, plus whatever produces the reward.
- **Policy** π — the agent's strategy: a (possibly probabilistic) mapping from state to action. For an LLM, the policy is exactly the model's next-token probability distribution — `softmax(logits)` from your own `04_run_tiny_gpt.py`. Training the model IS updating the policy.
- **Reward** — a scalar score for how good an outcome was. Not given per-token, usually — given for a whole completed response (did this answer actually help; was this proof actually correct).



For an LLM: agent = the model itself. action = which token to emit.
environment = the conversation so far. reward = a learned or verified score
of the FULL response (helpfulness, correctness) — not any single token.

Figure 3. The reinforcement learning loop. Unlike backprop against a fixed label (Part I), the "correct answer" here is only ever a scalar reward — the agent must discover which actions earn it.

7. Policy gradients: a worked bandit example

The simplest possible RL problem — a **multi-armed bandit** — has no state at all, just repeated choices among fixed actions with unknown rewards. It's enough to derive the core update rule used (in vastly scaled-up form) by PPO, GRPO, and every RLHF/RLVR pipeline: **increase the probability of actions that scored above average; decrease the probability of actions that scored below average.**

```
Policy:  $\pi(a) = \text{softmax}(\text{logits})_a$            two actions here, A and B

Policy gradient (REINFORCE): push logits in the direction that makes a GOOD action more likely
advantage = reward - baseline                    (baseline = a running average reward, e.g. from other samples)
d(logits) = advantage · ( one_hot(action_taken) -  $\pi$  )
logits_new = logits +  $\eta$  · d(logits)
```

Worked example — one policy-gradient update, two actions, computed live

Start: logits = [0. 0.] -> pi = softmax(logits) = [0.5 0.5] (uniform: no preference yet)

The agent samples action A (index 0) from this distribution, and the environment returns reward = 1.0.
Baseline (running average reward so far) = 0.5.

Step 1 - advantage: reward - baseline = 1.0 - 0.5 = 0.5
(positive => this action did BETTER than typical => reinforce it)

Step 2 - policy gradient:
d(logits) = 0.5 * ([1. 0.] - [0.5 0.5]) = 0.5 * [0.5 -0.5] = [0.25 -0.25]

Step 3 - update logits, learning rate 0.5:
logits_new = [0. 0.] + 0.5 * [0.25 -0.25] = [0.125 -0.125]

Step 4 - new policy:
pi_new = softmax([0.125 -0.125]) = [0.5622 0.4378]

Answer: P(A) rose from 0.500 to 0.5622 after ONE rewarded sample of A

No labeled "correct action" was ever provided — only a scalar reward after the fact. Scale this from 2 actions to a 151,936-token vocabulary, from one decision to a whole generated response, and from a hand-set baseline to a learned value function or a group-relative baseline (GRPO's contribution), and this exact update is what reshapes an LLM's behavior during RLHF and RLVR.

8. Why LLMs need RL — the bridge to RLHF and RLVR

You already built the supervised half of post-training by hand in `05_post_training_sft.py` : SFT clones curated example responses, with loss masked onto only the answer tokens — it is Part I's backpropagation, unchanged, just pointed at a small curated dataset instead of raw web text. SFT has a hard ceiling, though: it can only make the model imitate responses a human (or another model) already wrote down. It cannot directly optimize for "which of these two responses is better," because that comparison has no single token-by-token target to backpropagate against — there is a judgment, not a label.

Method	Training signal	What it optimizes
SFT	Curated (prompt, ideal response) pairs	Imitate the given response, token by token — supervised, Part I's machinery exactly.
RLHF (PPO)	A learned reward model, trained on human A/B preference judgments	Maximize predicted human preference — policy gradient (Part II), reward is learned, not given.
DPO	Human A/B preference pairs, directly	Same goal as RLHF, without training a separate reward model or running RL rollouts — a clever reformulation, not covered further here.
RLVR (GRPO)	An automatic verifier (does the code run; is the proof correct)	Maximize verified correctness on checkable tasks — reward has no human in the loop at all, which is why RLVR can run at far larger scale than RLHF.

Try this — feel RLVR's fingerprint directly

Ask a "reasoning" model (trained with RLVR, e.g. anything advertising extended/deep thinking) a moderately hard math word problem, then ask a same-generation NON-reasoning chat model the exact same question with "answer directly, no working."

Why this shows it: RLVR's reward only ever checks the FINAL answer, never the reasoning text itself — yet models trained this way spontaneously produce long, self-correcting chains of reasoning before answering, because that's what the reward landscape rewarded discovering. That behavior is absent in a model that only ever saw SFT/RLHF on short, direct answers — it's a direct behavioral trace of which training method produced the model.

CHAIN → RL tells you how to shape behavior toward a scalar reward. But everything so far has assumed a fixed-size input — a vector of numbers, a single bandit choice. Text is a variable-length SEQUENCE, and the next architectural problem is: how do you build a network that reads a sentence, keeping track of relationships between words arbitrarily far apart? Part III.

PART III

THE TRANSFORMER, FROM SCRATCH

Self-attention, multi-head splitting, positional encoding, and why GPT dropped half the original architecture.

9. Why recurrence wasn't enough

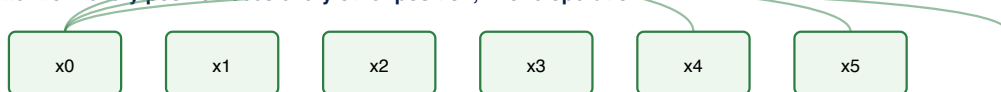
Before 2017, sequence models (RNNs, LSTMs) read text one token at a time, carrying a running "hidden state" forward — token 50's representation depends on token 49's, which depends on token 48's, and so on back to the start. Two consequences follow directly from that design: **training cannot parallelize across the sequence** (you must finish step $t-1$ to start step t), and **information from far-away tokens has to survive being repeatedly compressed** through every intermediate step, which in practice makes long-range dependencies hard to learn. "Attention Is All You Need" (Vaswani et al., 2017) proposed removing recurrence entirely.

RNN: sequential, one token at a time



h_t depends on $h_{(t-1)}$ — must finish step $t-1$ before starting t . Long-range signal fades over many steps.

Attention: every position sees every other position, in one operation

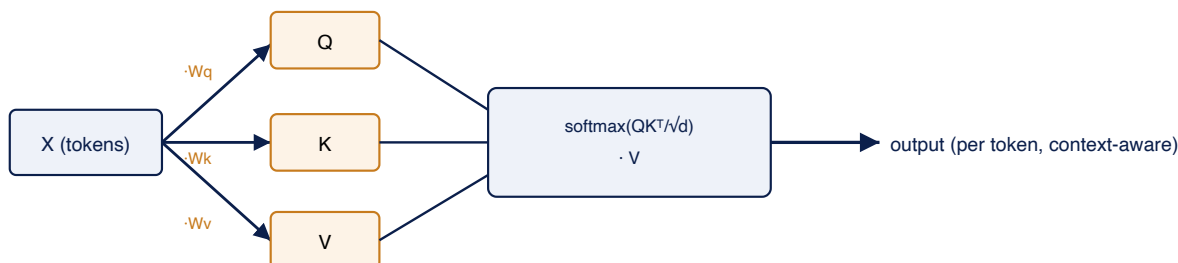


Every x_i can attend directly to every x_j in a single matmul — no waiting, no fading over distance.

Figure 4. The problem attention solves. RNNs process a sequence step by step, so training cannot parallelize across time and distant tokens must survive being compressed through every intermediate hidden state. Attention computes all pairwise relationships in one shot.

10. Self-attention with learned projections

Your own `02_attention_from_scratch.py` demonstrated the raw mechanism by setting $Q=K=V=X$ directly, with a comment flagging that real models use learned projections "in step 4." This is step 4. Three separate weight matrices, learned during training, project the same input three different ways before the identical $\text{softmax}(QK^T/\sqrt{d}) \cdot V$ machinery runs.



Q asks "what am I looking for"; K advertises "what do I contain"; V is "what I'll actually contribute" — three DIFFERENT learned projections of the same input, each playing a different role in the same operation.

Figure 5. Query/Key/Value attention. Your `02_attention_from_scratch.py` used $Q=K=V=X$ directly to expose the raw mechanism; the worked example below adds the learned W_q , W_k , W_v projections it deferred.

Worked example — re-running your river/bank/loan disambiguation with LEARNED (not identity) projections

Same 4-dim toy embeddings as 02_attention_from_scratch.py. Projections (diagonal, for readability — amplifies the "finance-ness" dimension, dim index 1, for both queries and keys):

```
Wq = Wk = diag([0.5, 2.5, 0.5, 0.5]), Wv = identity
```

```
Sentence A: ['the', 'river', 'bank']
Q = X @ Wq =
[[0.  0.  0.5  0. ]
 [0.5  0.  0.  0. ]
 [0.15 0.75 0.  0. ]]
K = X @ Wk =
[[0.  0.  0.5  0. ]
 [0.5  0.  0.  0. ]
 [0.15 0.75 0.  0. ]]
scores = Q @ K.T / sqrt(4) =
[[0.125  0.  0.  ]
 [0.  0.125 0.0375]
 [0.  0.0375 0.2925]]
softmax row for "bank" -> weights = [0.296  0.3073 0.3966]
"bank" output vector = [0.4263 0.119  0.296  0.  ]
distance to river embedding = 0.6564 distance to loan embedding = 1.0225

Sentence B: ['the', 'loan', 'approved', 'bank']
"bank" output vector = [0.0573 0.6632 0.1426 0.302 ]
distance to river embedding = 1.2000 distance to loan embedding = 0.4778
```

Answer: in the river sentence, bank sits closer to river (0.656 vs 1.023); in the finance sentence, it flips to sit closer to loan (0.478 vs 1.200)

Identical disambiguation result to your original identity-projection demo — the learned $W_q/W_k/W_v$ change *what gets emphasized* (here, deliberately, the finance-vs-nature dimension) but not the underlying mechanism. In a trained model, $W_q/W_k/W_v$ are learned by exactly the backpropagation of Part I, discovering on their own which dimensions matter for which relationships.

11. Multi-head attention

One attention pattern can only capture one kind of relationship at a time — whatever the single Q/K projection happens to emphasize. Multi-head attention runs several smaller attention operations in parallel, each with its own slice of the projected space, then concatenates the results — letting different heads specialize in different kinds of relationships simultaneously.

Worked example — splitting the 4-dim projections above into 2 heads of width 2

Head 1 uses dims [0,1] (the "nature-ness / finance-ness" subspace):

```
Q1,K1,V1 for "bank" = [0.15 0.75], [0.15 0.75], [0.3 0.3]
attention weights (bank row) = [0.2804 0.2956 0.424 ]
head 1 output (bank row) = [0.4228 0.1272]
```

Head 2 uses dims [2,3] (the "definite-ness / verb-ness" subspace):

```
Q2,K2,V2 for "bank" = [0. 0.], [0. 0.], [0. 0.]
attention weights (bank row) = [0.3333 0.3333 0.3333]
head 2 output (bank row) = [0.3333 0.  ]
```

Concatenate the two heads' outputs back together:

```
concat(head1, head2) = [0.4228 0.1272 0.3333 0.  ]
```

Answer: concatenated multi-head output = [0.4228 0.1272 0.3333 0.]

Honest limitation of this toy: head 2's weights come out perfectly uniform (0.333 each) because none of these three tokens differ at all in the "definite/verb" subspace it's using — there's no signal for it to find. In a real model, with hundreds of dimensions per head and a huge vocabulary that genuinely varies along every one of them, different heads reliably specialize — some track syntax, some track coreference, some track long-range topic. This is exactly the Q: 14 heads / K,V: 2 heads split you already read about in the Qwen2.5 Architecture document's GQA section, before GQA's extra trick of sharing K/V across groups of heads.

12. Positional encoding, the original sinusoidal form

Attention as defined above is **order-blind** — $\text{softmax}(QK^T)V$ gives the same answer no matter what order the tokens are fed in, since nothing in the formula references position. The original Transformer fixed this by adding a deterministic, non-learned "position signal" to each token's embedding before the first layer — different frequencies of sine and cosine waves, one pair of dimensions per frequency.

```
PE(pos, 2i)   = sin( pos / 10000^(2i/d_model) )
PE(pos, 2i+1) = cos( pos / 10000^(2i/d_model) )    i indexes dimension-pairs, same base-10000 idea RoPE later reused
```

Worked example — positional encoding vectors at four different positions, $d_{\text{model}}=4$

```
PE(pos=0) = [0. 1. 0. 1.]
PE(pos=1) = [0.8415 0.5403 0.01 1. ]
PE(pos=2) = [ 0.9093 -0.4161 0.02 0.9998]
PE(pos=50) = [-0.2624 0.965 0.4794 0.8776]
```

Answer: each position gets a distinct, fixed vector, added directly to that position's token embedding

This is GPT-2's `wpe` made non-learned and formulaic instead of a trainable table — same role (inject position), different mechanism (fixed sinusoid vs. learned rows). Note the family resemblance to RoPE's $\text{base}^{(-2i/d)}$ frequency spread from the Qwen2.5 document — RoPE is best understood as this same idea, relocated from "added once at the input" to "applied as a rotation inside every attention layer."

13. The encoder block, assembled

One encoder layer, exactly as published in 2017: multi-head self-attention, a residual connection, LayerNorm, a position-wise feed-forward network (two linear layers with a ReLU between them — literally GPT-2's MLP, one paper earlier), another residual connection, another LayerNorm. Six of these stacked make the encoder.

```
FFN(x) = max(0, x·W1 + b1)·W2 + b2          — ReLU, not GELU; GPT-2 (2019) swapped in GELU

EncoderLayer(x):
    x = LayerNorm(x + MultiHeadAttention(x, x, x))    (self-attention: Q,K,V all from the same x)
    x = LayerNorm(x + FFN(x))
```

14. Decoder-only: why GPT dropped the encoder

The original decoder layer has *three* sub-blocks: masked (causal) self-attention, cross-attention into the encoder's output, then the feed-forward network — needed because machine translation has two different sequences (source, target) that must interact. Radford et al.'s GPT (2018) and GPT-2 (2019) noticed that framing nearly any task as "predict the next token of one sequence" removes the need for a second sequence at all — delete the encoder, delete cross-attention, keep only masked self-attention and the FFN. What remains is exactly the block you already trained in `03_train_tiny_gpt.py`.

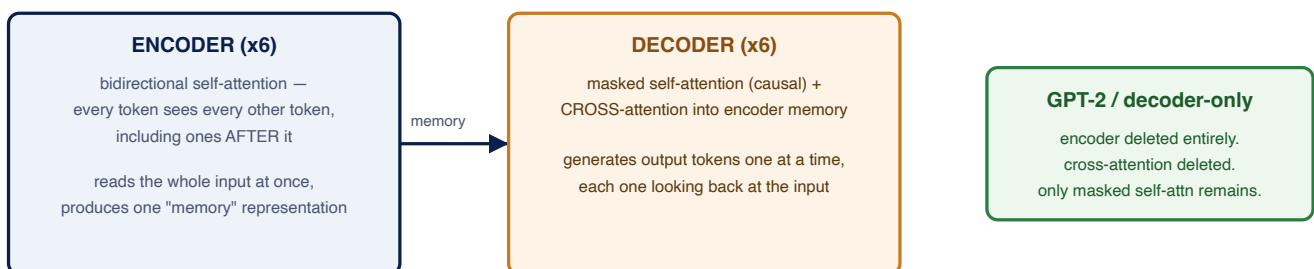


Figure 6. The 2017 Transformer was built for translation: an encoder reads the source sentence, a decoder generates the target sentence while attending back to it. GPT-2 keeps only the decoder half, with nothing left to cross-attend into — which is exactly the architecture you already built.

Try this — feel the encoder-only vs. decoder-only distinction

Compare a BERT-family model (encoder-only, e.g. any "fill-mask" demo) filling in "The capital of France is [MASK]." against a GPT-family model completing "The capital of France is ____".

Why this shows it: BERT's encoder can see BOTH sides of the blank simultaneously (full bidirectional attention — good for understanding/classification, unable to generate left-to-right text at all). GPT's decoder can only see what came before — worse at some classification tasks, but this exact restriction is what makes open-ended generation possible one token at a time.

CHAIN → The Transformer, GPT-2-style, is now fully derived from first principles. Part IV is a short checkpoint — connecting every piece above to the actual GPT-2 you already inspected — before Part V asks: which of GPT-2's design choices turned out to have hidden costs at greater scale, and what replaced them?

PART IV

GPT-2: CHECKPOINT

You already built and verified every piece of this. This chapter is the map back to your own files, not a re-derivation.

15. GPT-2, recapped against what you already built

Delete the encoder and cross-attention from Part III's Figure 6, keep the decoder's masked self-attention and feed-forward network, swap ReLU for GELU, and stack the result N times. That is GPT-2 — and it is, component for component, what you built and ran yourself:

GPT-2 component	Where you already built and verified it
Token embedding (wte) + learned position embedding (wpe)	<code>TinyGPT.__init__</code> in <code>03_train_tiny_gpt.py</code> ; Part III Section 12 gave the fixed-formula version this generalizes from.
Masked multi-head self-attention	<code>02_attention_from_scratch.py</code> (raw mechanism) → Part III Sections 10–11 of this document (learned projections, multi-head split) → <code>TinyGPT.forward</code> 's attention block.
Residual connections, pre-norm, GELU MLP	<code>TinyGPT.forward</code> and <code>04_run_tiny_gpt.py</code> 's <code>x = x + out / gelu(...)</code> lines.
Weight tying (wte doubles as the un-embedding matrix)	<code>logits = x @ self.wte.T</code> in both your <code>TinyGPT</code> and <code>04_run_tiny_gpt.py</code> .
Training: next-token cross-entropy, backpropagation, gradient descent	Exactly Part I of this document, run for real in <code>03_train_tiny_gpt.py</code> (2,000 steps, AdamW).
The real, production 124M-parameter version, inspected directly	<code>06_inspect_real_model.py</code> — downloaded, ran, and diffed against your own model's parameter names and shapes.

What Part IV deliberately does not repeat

Every formula, every worked matrix multiplication, and every diagram for this exact architecture already exists in your own lab and was verified by actually running the code and reading the real output — repeating it here would be transcription, not teaching. If any piece above feels shaky, the fix is to re-open that file and re-run it, not to re-read prose about it.

CHAIN → GPT-2 (2019) trained at 124M–1.5B parameters on ~40GB of text with a 1,024-token context window. Scaling the exact same architecture up by 1,000× and the context window out by 30–100× exposed costs in GPT-2's design that were invisible at its original scale — Part V derives each fix, one at a time.

PART V

WHAT CHANGED AFTER GPT-2

Four substitutions, each solving one specific cost of GPT-2's design that only became visible at greater scale. Condensed from the companion Qwen2.5 Architecture document — read that one for the full code-alongside-explanation treatment of each; this chapter keeps every formula and worked example, in the chain's own sequence.

16. RMSNorm — the cost of centering

What GPT-2 paid for: LayerNorm subtracts the mean AND rescales by the variance, at every one of 2 norms per layer, every layer, every token, every training step. At GPT-2's 12 layers this is negligible; at 94 layers (DeepSeek-V3) it is not. RMSNorm found empirically that the mean-subtraction step contributes nothing to stability — only the rescaling matters.

```
RMSNorm(x) = ( x / sqrt(mean(x2) + ε) ) · g — no mean, no bias term
```

qwen2_modeling_reference.py

```
class Qwen2RMSNorm(nn.Module):
    def forward(self, hidden_states):
        variance = hidden_states.pow(2).mean(-1,
        keepdim=True)
        hidden_states = hidden_states *
        torch.rsqrt(variance + self.variance_epsilon)
        return self.weight * hidden_states
```

No `x.mean(-1)` anywhere — that's the whole change. One fewer reduction op, computed billions of times over a training run, at effectively zero quality cost.

Worked example

```
x = [ 2.  -1.   0.5  3.  -2.5  1. ], g = [1.  1.  1.2  1.  0.8  1. ], eps = 1e-06
mean(x^2) = 3.583333 -> rms = sqrt(3.583333+1e-06) = 1.892970
y = (x/rms)*g = [ 1.0565 -0.5283  0.317  1.5848 -1.0565  0.5283]
```

Answer: `y = [ 1.0565 -0.5283 0.317 1.5848 -1.0565 0.5283]`

17. RoPE — the cost of a fixed position table

What GPT-2 paid for: a learned `wpe` table has exactly as many rows as its context window — GPT-2 hard-caps at 1,024 tokens, structurally, because row 1,025 does not exist. RoPE replaces the table with a formula: rotate Q and K by an angle proportional to position, inside every attention layer. No learned parameters, no upper bound baked into a table size.

```
θi = base-(2i/d) rotate pair [xi, x(i+d/2)] by angle m·θi at position m
Key fact: (R(m)q) · (R(n)k) depends ONLY on (n-m) — relative position, never absolute
```

qwen2_modeling_reference.py

```
def rotate_half(x):
    x1, x2 = x[...], x.shape[-1]//2, x[...],
    x.shape[-1]//2:]
    return torch.cat((-x2, x1), dim=-1)

def apply_rotary_pos_emb(q, k, cos, sin):
    q_embed = (q * cos) + (rotate_half(q) * sin)
    k_embed = (k * cos) + (rotate_half(k) * sin)
    return q_embed, k_embed
```

Applied to Q and K only, freshly, inside every layer — not once at the input like `wpe`. This is why a model trained mostly on shorter sequences can still be extended to much longer context (with techniques like YaRN scaling) — there is no table to run out of rows in.

Worked example — relative-position invariance, proved numerically

```
head_dim=8, base=1,000,000, inv_freq=[1. 0.0316 0.001 0. ]
m= 2, n= 5 (n-m=3) -> q_rot(m).k_rot(n) = -3.722394
m=10, n=13 (n-m=3) -> q_rot(m).k_rot(n) = -3.722394
m=100, n=103 (n-m=3) -> q_rot(m).k_rot(n) = -3.722394
```

Answer: all three identical (-3.722394) despite wildly different absolute positions

Try this — feel RoPE's practical consequence

Paste a ~20,000-word document into a long-context model and ask it to quote one specific sentence from near the very beginning.

Why this shows it: GPT-2's `wpe` table would make this request structurally impossible past token 1,024 — there is no position vector for token 15,000. A RoPE-based model has no such wall; whether it succeeds is a separate question about training and attention quality, but it is at minimum ABLE to represent the position at all.

18. Grouped-Query Attention — the cost of one K/V per head

What GPT-2 paid for: every attention head caching its own K and V means the KV-cache — the memory that accumulates as a conversation grows — scales with the number of heads. GQA asks whether every head truly needs its own K/V, and finds the answer is no: Qwen2.5-0.5B shares one K/V head across every 7 query heads.

```
qwen2_modeling_reference.py
def repeat_kv(hidden_states, n_rep):
    b, n_kv, s, d = hidden_states.shape
    hidden_states = hidden_states[:, :, None, :,
    :].expand(b, n_kv, n_rep, s, d)
    return hidden_states.reshape(b, n_kv * n_rep, s, d)

self.num_key_value_groups = num_attention_heads //
num_key_value_heads
self.k_proj = nn.Linear(hidden_size, num_key_value_heads
* head_dim, bias=True) # SMALLER than q_proj
```

k_proj/v_proj output a SMALLER dimension than q_proj — that one shape difference is the entire mechanism. `repeat_kv` duplicates each real KV head just before the matmul so shapes still line up; only the smaller, unduplicated version is ever cached between generation steps.

Worked example

```
4 query heads, 2 KV heads -> group size 2
K before repeat (2x3):
[[ 0.346  0.822  0.33 ]
 [-1.303  0.905  0.446]]
K after repeat_kv (4x3):
[[ 0.346  0.822  0.33 ]
 [ 0.346  0.822  0.33 ]
 [-1.303  0.905  0.446]
 [-1.303  0.905  0.446]]
```

Real Qwen2.5-0.5B, per token, both K&V, all 24 layers:
full MHA cache = $14 \times 64 \times 2 \times 24 = 43,008$
GQA cache = $2 \times 64 \times 2 \times 24 = 6,144$

Answer: 7.0x smaller KV-cache, every generated token

Try this — feel GQA's practical consequence

Hold a very long back-and-forth conversation (50+ turns) with a model and watch response latency as the conversation grows, versus restarting a fresh conversation each time.

Why this shows it: latency growth with conversation length is dominated by KV-cache size — the more heads' worth of K/V must be read back for every new token, the more that growth costs. GQA (and MLA, Section 21) is precisely what keeps that curve flatter than it would otherwise be.

19. SwiGLU — the cost of a fixed nonlinearity

What GPT-2 paid for: GELU applies the identical fixed curve to every feature, every token, no matter what. SwiGLU adds a second, parallel projection that acts as a learned, per-feature, per-token gate — the network decides how much of each feature to let through, rather than a fixed function deciding for it.

GPT-2: `down( GELU( up(x) ) )`

SwiGLU: `down( SiLU(gate(x)) · up(x) )`

```
qwen2_modeling_reference.py
class Qwen2MLP(nn.Module):
    def forward(self, x):
        return
self.down_proj(self.act_fn(self.gate_proj(x)) *
self.up_proj(x))
```

Two "up" projections instead of one — gate and up are separate learned matrices; their elementwise product (not a matmul) feeds the down-projection. Consistently lower loss per parameter than GELU, at the cost of a third weight matrix.

Worked example

```
x=[ 0.189 -0.523 -0.413 -2.441  1.8    1.144]
gate = x@Wg = [ 0.9389  0.6956 -2.9265 -0.1038] -> silu(gate) = [ 0.675    0.4641 -0.1488 -0.0492]
up = x@Wu = [ 1.5695  1.7236 -0.812  -0.004 ]
fused = silu(gate)*up = [1.0594 0.8    0.1209 0.0002]
out = fused@Wd = [ 0.2133  0.3043  1.2587  0.1071  0.2028 -0.8356]
```

Answer: out = [0.2133 0.3043 1.2587 0.1071 0.2028 -0.8356]

Honest limitation: unlike RoPE or GQA, SwiGLU has no single-prompt behavioral fingerprint — it's a quality/efficiency improvement in how the MLP represents information, not a capability toggle you can elicit with one input. Not every architectural improvement is directly observable this way, and it's worth knowing which kind you're looking at.

CHAIN → RMSNorm, RoPE, GQA and SwiGLU take GPT-2 all the way to Qwen2.5 — a dense model, meaning every parameter is used on every token. The next wall is dense compute itself: making a model bigger makes EVERY token pay for the whole thing. Part VI is 2026's answer.

PART VI

2026 STATE OF THE ART

Mixture-of-Experts and Multi-Head Latent Attention — grounded in DeepSeek-V3's real, published config and code, no weights downloaded.

20. Mixture-of-Experts (MoE) — the cost of a dense MLP

What every model so far paid for: in Part V's SwiGLU MLP, every single token is pushed through the *entire* MLP — the same gate/up/down matrices, whether the token is "the" or the hardest word in the sentence. Making the model smarter by making that MLP bigger means EVERY token now costs more, even the easy ones. MoE breaks this coupling: replace one large MLP with many smaller "expert" MLPs, and route each token to only a handful of them.

```
router_logits = x · W_router          one score per candidate expert
top_k experts selected by score, their outputs combined by (normalized) score-weighted sum
+ a "shared expert" that runs for EVERY token unconditionally, regardless of routing
```

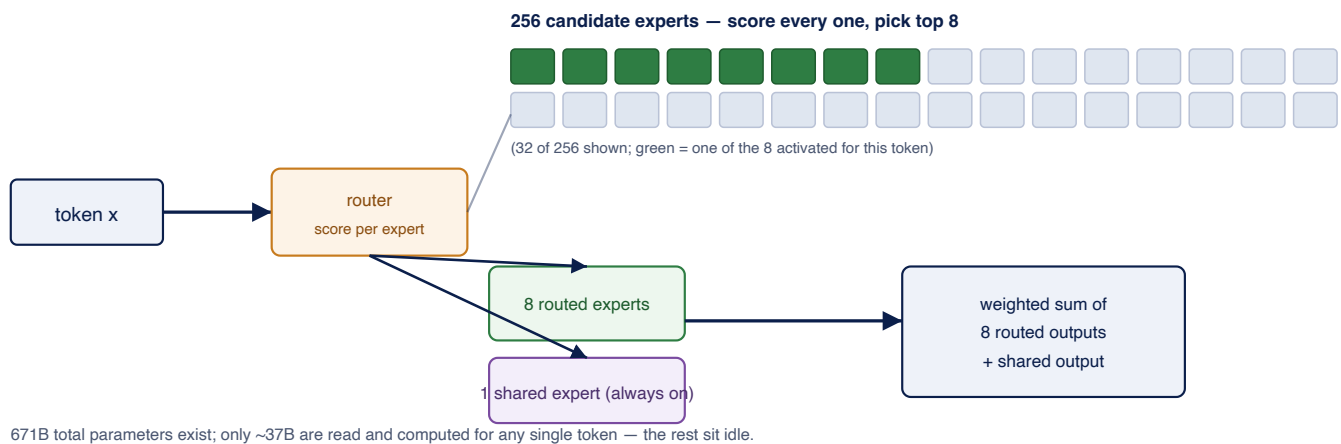


Figure 7. DeepSeek-V3's MoE layer. 256 candidate experts hold the model's total knowledge; only 8 plus 1 always-on shared expert actually run for any given token — decoupling total capacity from per-token compute cost.

```
modeling_deepseek_v3.py (trimmed — real V3 code adds a group-based
masking step for deployment load-balancing, omitted here; n_group=8,
topk_group=4 in the real config)

class DeepseekV3TopkRouter(nn.Module):
    def forward(self, hidden_states):
        router_logits = F.linear(hidden_states,
            self.weight)
        scores = router_logits.sigmoid()
        topk_indices = torch.topk(scores, k=self.top_k,
            dim=-1)[1]
        topk_weights = scores.gather(1, topk_indices)
        topk_weights /= topk_weights.sum(dim=-1,
            keepdim=True)
        return topk_weights, topk_indices

class DeepseekV3MoE(nn.Module):
    def forward(self, hidden_states):
        residuals = hidden_states
        topk_weights, topk_indices =
            self.gate(hidden_states)
        routed_out = self.experts(hidden_states,
            topk_indices, topk_weights)
        # shared expert: same MLP, EVERY token, no
        routing decision
        return routed_out +
            self.shared_experts(residuals)
```

router_logits = F.linear(x, W_router) — one linear layer, output dimension = number of experts (256 in V3). This is the ENTIRE routing decision, per token, recomputed fresh every layer — cheap compared to running even one expert MLP.

scores.sigmoid(), not softmax — each expert gets an independent 0-1 affinity score rather than a distribution that must sum to 1 across all 256; only after top-k selection are the chosen few renormalized to sum to 1 among themselves.

the shared expert is not routed at all — it is architecturally always-on, carrying whatever general-purpose computation every token needs regardless of specialization, while the routed experts specialize on top of that common baseline.

Worked example — 4 candidate experts, top-2 routing, plus 1 shared expert, computed live

```
x = [ 2.041 -2.556  0.418 -0.568]
router weight W_r (one row per expert) =
[[-0.272 -0.129 -1.212 -0.139]
 [-0.519  1.994  0.135 -0.212]
 [-0.169 -0.401 -0.633 -0.234]
 [ 0.289 -0.143  0.575 -0.12 ]]
```

Step 1 - router logits = $W_r @ x$:

```
logits = [-0.6531 -5.9791  0.5483  1.2639]
```

Step 2 - sigmoid score per expert:

```
scores = [0.3423 0.0025 0.6338 0.7797]
```

Step 3 - pick top-2 experts by score:

```
chosen experts (0-indexed) = [3, 2], scores = [0.7797 0.6338]
```

Step 4 - renormalize chosen scores to sum to 1:

```
weights = [0.5516 0.4484]
```

Step 5 - each expert's own tiny transform of x (only the CHOSEN ones actually get computed in real MoE - all 4 shown here so you can see what was skipped):

```
expert0(x) = [-1.3533 -0.3162 -1.3049 -0.0093]  <- NOT computed for this token in real MoE
expert1(x) = [-3.1354 -0.3187  0.1392 -0.8023]  <- NOT computed for this token in real MoE
expert2(x) = [ 0.0239  2.7421 -1.7362 -0.8701]  <- chosen
expert3(x) = [-1.0594  2.8121  1.498  0.4716]   <- chosen
```

Step 6 - weighted combination of the CHOSEN experts only:

```
routed = 0.5516*expert3(x) + 0.4484*expert2(x) = [-0.5737  2.7807  0.0478 -0.13  ]
```

Step 7 - add the always-on shared expert:

```
shared(x) = [ 0.7411  1.9381 -0.1674 -0.8745]
final = routed + shared = [ 0.1674  4.7188 -0.1196 -1.0045]
```

Answer: final MoE output = [0.1674 4.7188 -0.1196 -1.0045], computed from 3 of 4 candidate expert MLPs (2 routed + 1 shared) — expert1 was never touched for this token

Real DeepSeek-V3 numbers (from config.json, no weights downloaded)

Total parameters	671,000,000,000
Activated per token	37,000,000,000 (5.5% of total)
Routed experts (candidates)	256
Routed experts activated/token	8
Shared experts (always on)	1
Each expert's width (moe_intermediate_size)	2048 (vs 7168 hidden_size — each expert is deliberately narrow)
First dense (non-MoE) layers	3 of 61 — early layers stay dense, MoE only kicks in from layer 3 onward

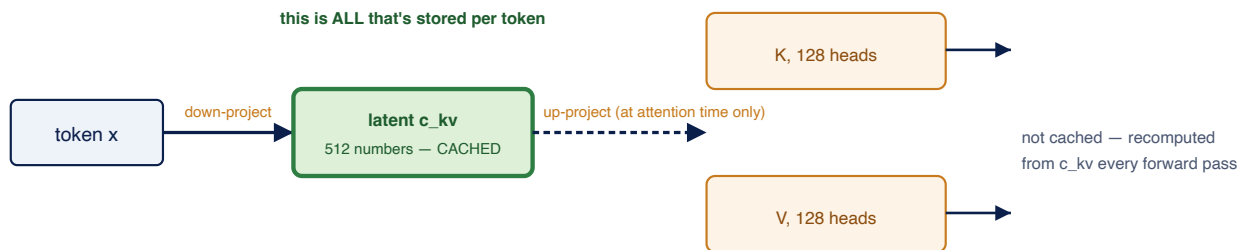
Try this — probe for MoE's fingerprint

In one prompt, ask a MoE-based model to (a) write a short poem, then immediately (b) debug a snippet of Python, then (c) translate a sentence to French — three very different skills back to back.

Why this shows it: there's no way to directly observe which of 256 experts fired from the outside — but the underlying hypothesis MoE architectures are built on is that different experts specialize on different kinds of sub-tasks. A model handling an abrupt hard context-switch between unrelated skill domains without degradation is indirect (not conclusive) evidence the routing is doing real specialization work, not just noise.

21. Multi-Head Latent Attention (MLA) – compressing the cache further still

What GQA still paid for: GQA shrinks the KV-cache by sharing K/V *across* groups of heads — but every head that does have its own K/V still stores it at full size. MLA takes a different approach entirely: don't shrink the number of distinct K/V "slots" at all — compress WHAT gets cached into a small shared latent vector, and reconstruct full per-head K/V from it only at the moment attention actually needs them, never storing the reconstructed version.



GQA (Part V) shrinks the cache by sharing K/V ACROSS heads. MLA shrinks it differently: every head still gets its own K/V, but neither is EVER stored in full size — only the small latent is.

Figure 8. Multi-Head Latent Attention. The 40,960 numbers a full 128-head MHA cache would need, per token per layer, collapse to a 512+64=576-number latent — reconstructed to full per-head size only inside the attention computation itself, never written to the cache.

```
modeling_deepseek_v3.py, DeepseekV3Attention (trimmed)
# down-project: hidden_size -> a much smaller compressed
latent
self.kv_a_proj_with_mqa = nn.Linear(hidden_size,
kv_lora_rank + qk_rope_head_dim, bias=False)
self.kv_a_layernorm = DeepseekV3RMSNorm(kv_lora_rank)
# up-project: latent -> full per-head size (computed
fresh, never cached)
self.kv_b_proj = nn.Linear(kv_lora_rank, num_heads *
(qk_nope_head_dim + v_head_dim), bias=False)

def forward(self, hidden_states, position_embeddings,
past_key_values=None):
    compressed_kv =
self.kv_a_proj_with_mqa(hidden_states)
    kv_nope, k_rot = torch.split(compressed_kv,
[kv_lora_rank, qk_rope_head_dim], dim=-1)
    kv_nope = self.kv_a_layernorm(kv_nope)
    if past_key_values is not None:
        # ***cache read/write happens HERE - on the small
compressed latent***
        kv_nope, k_rot = past_key_values.update(kv_nope,
k_rot, self.layer_idx)
    # up-project back to full size only when attention
actually runs, this step
    key_states, value_states = self.expand_kv(kv_nope,
k_rot)
```

kv_a_proj_with_mqa — the down-projection. "mqa" in the name is a hint: this one small projection is shared across all heads (multi-query-attention-style), producing a SINGLE small latent per token, not one per head.

past_key_values.update(kv_nope, k_rot, ...) happens on the COMPRESSED latent — this is the line that determines what actually lives in GPU memory as the conversation grows. Compare this to GQA's

`past_key_values.update(key_states, value_states, ...)` in the Qwen2.5 code, which caches full-size (if fewer) K/V tensors.

kv_b_proj — the up-projection, applied fresh every single forward pass, right before the attention matmul. This is deliberate extra compute traded for a much smaller memory footprint — a classic compute-for-memory trade, worthwhile because memory bandwidth, not raw FLOPs, is usually the binding constraint during generation.

Why RoPE needs special handling here: compressing a vector that's already been rotated by RoPE and then reconstructing it later does not commute cleanly with the rotation. DeepSeek's fix (visible in the split into `kv_nope / k_rot`) is to keep a small separate, uncompressed, shared "rope-carrying" component alongside the compressible content — decoupled RoPE.

Worked example — toy down-project / up-project cycle, computed live

hidden_size=8, 4 heads x head_dim=2, kv_lora_rank=4, rope_dim=2

Step 1 - down-project x into the small latent (THIS gets cached):

```
c_kv = x @ W_down_kv = [-1.5983  1.3592 -0.1657 -0.8609]      (4 numbers)
k_rope = x @ W_down_rope = [-1.8505 -0.1335]      (2 numbers, shared across all heads)
cache size this token = 6 numbers      (vs 16 for full per-head K+V in this toy)
```

Step 2 - up-project the latent back to full per-head size (NOT cached, recomputed every attention call):

```
K_rope (per head) = (c_kv @ W_up_K).reshape(4,2) =
[[ 1.2447 -0.801 ]
 [-0.5683 -0.3536]
 [-0.5526 -1.4382]
 [ 0.7611 -0.0059]]
V (per head) = (c_kv @ W_up_V).reshape(4,2) =
[[ 0.8051  1.515 ]
 [ 0.9024  0.2984]
 [ 0.0067  0.6096]
 [-0.3708  0.9029]]
```

Real DeepSeek-V3 (from config.json): kv_lora_rank=512, qk_rope_head_dim=64
MLA cache/token/layer = 512 + 64 = 576
full-MHA-equivalent (128 heads, head dims from config) = 40,960
ratio = 71.1x smaller

Answer: 71.1x smaller cache per token, computed directly from DeepSeek-V3's real config — separately, DeepSeek-V2's own paper reports a 93.3% KV-cache reduction versus their earlier dense 67B model, a different (paper-chosen) baseline comparison, same underlying mechanism

Try this — same fingerprint as GQA, more pronounced

Feed a model with a very large context window (100K+ tokens) a long document and ask several follow-up questions in the same conversation, timing responses as the conversation grows.

Why this shows it: MLA and GQA are both answers to the exact same observable symptom (latency and memory growth with conversation length) — you cannot tell from outside a chat interface which specific mechanism a given model uses, but the qualitative signature (long contexts staying usable at all) is the direct payoff of whichever compression technique is underneath.

22. The landscape beyond attention

Two more directions worth knowing exist, kept brief deliberately — full derivation would roughly double this document's length for a payoff outside its main chain:

- **Sliding-window / hybrid attention** — restrict most layers to attending only within a local window (cheap, linear in sequence length) and let a minority of layers do full attention, trading a little long-range precision for a large compute saving. Several 2025-2026 open models interleave these two layer types.
- **State-space models (Mamba and hybrids)** — a genuinely different sequence-mixing primitive, not attention at all, with compute that scales linearly rather than quadratically with sequence length. As of 2026 these mostly appear as hybrids (a few state-space layers mixed with a few attention layers) rather than wholesale replacements — attention's ability to look back at any exact position with no decay still wins on tasks needing precise long-range recall.

CHAIN → MoE decouples total knowledge from per-token cost; MLA decouples cache size from head count. Neither changes what the model is trained to do — Parts I and II's backpropagation and policy-gradient machinery, applied at enormous scale, are still the entire training story. Part VII closes the loop.

PART VII

CLOSING

The whole chain on one page, every prompt collected in one place, and where each mechanism actually lives on your Mac.

23. The whole chain, one page

Stage	What it could do	What broke	What fixed it
Single neuron	Weighted vote → one straight-line decision boundary	Can't represent curves, XOR, anything non-linear	Stack neurons in layers with a nonlinearity between them (Part I.2)
Deep network	Represents arbitrary functions, in principle	No way to know which direction millions of weights should move	Backpropagation: the chain rule, applied mechanically (Part I.5)
Backprop against labels	Learns anything with a known correct answer	"Helpful and harmless" has no single correct label	Reinforcement learning: optimize a scalar reward instead (Part II)
Fixed-size vector networks	Classify, regress on vectors	Text is variable-length; RNNs process it sequentially — slow, and long-range signal fades	Self-attention: one parallel operation over all positions (Part III.9-10)
Single attention pattern	Captures one kind of token relationship	Different relationships (syntax, coreference, topic) need different attention patterns simultaneously	Multi-head attention (Part III.11)
Encoder-decoder Transformer	Translation and other seq-to-seq tasks	Needs two sequences; most tasks can be framed as "predict the next token of one"	Decoder-only: GPT-2 deletes the encoder (Part III.14)
GPT-2's LayerNorm, wpe, MHA, GELU MLP	Works well at 124M params, 1,024-token context	Each carries a cost invisible at that scale: extra reduction ops, a hard position ceiling, a KV-cache that scales with heads, a fixed nonlinearity	RMSNorm, RoPE, GQA, SwiGLU (Part V)
GQA's shared-but-full-size K/V	7x smaller cache than full MHA	Still stores full head-dimension K/V for every distinct KV head, at frontier model sizes this is still large	MLA: compress to a shared low-rank latent, reconstruct only when needed (Part VI.21)
Dense MLP (SwiGLU)	One MLP, applied identically to every token	Every token pays for the FULL MLP regardless of difficulty — the only way to add knowledge is to make every token more expensive	Mixture-of-Experts: route each token to a handful of specialists (Part VI.20)

24. Prompt library, collected

Architecture feature	Prompt to try	What it's evidence of
Encoder- vs decoder-only	Fill-mask on a BERT-style model vs. left-to-right completion on a GPT-style model, same sentence	Bidirectional vs. causal attention — Part III.14
RLVR / reasoning training	A hard math word problem, reasoning model vs. direct-answer model, "answer directly, no working" on the latter	Reward shaped only on final-answer correctness still produces spontaneous chain-of-thought — Part II.8
RoPE / long context	~20,000-word document, ask for a quote from near the start	No learned position table to run out of rows in — Part V.17
GQA / KV-cache	50+ turn conversation, watch latency growth vs. a fresh conversation	Smaller per-token cache growth — Part V.18
MoE / expert specialization	Poem, then code debugging, then translation, back to back in one prompt	Indirect evidence of per-subtask routing specialization — Part VI.20
MLA / extreme long context	100K+ token document, multiple follow-up questions in one conversation	Same fingerprint as GQA, more pronounced — Part VI.21

None of these prompts *prove* a specific architecture is in use behind an API — you cannot inspect the weights of a closed model this way. They demonstrate the CONSEQUENCE each architectural choice was built to produce, which you can then go verify mechanically, on open weights, using the code and worked examples throughout this document.

25. Glossary & where each piece lives on your Mac

Backpropagation

Computing dLoss/dWeight for every weight via repeated chain-rule application, back to front through the network. Part I.5.

Policy gradient

Nudging action probabilities toward whatever scored above a baseline reward, in proportion to how far above. Part II.7.

KV-cache

Stored key/value vectors from prior tokens, reused instead of recomputed at each generation step. What GQA (Part V.18) and MLA (Part VI.21) each shrink, by different mechanisms.

Router / gate (MoE)

The small linear layer that scores which experts a token should be sent to. Part VI.20.

Low-rank latent (MLA)

A compressed vector that a small down-projection produces and a matching up-projection can approximately reconstruct from — cheaper to store than the full-size original. Part VI.21.

File	What it covers
LLM_Internals_Lab/01-06	Tokenization, hand-written attention, a real trained tiny GPT (pretrain + SFT), and a real downloaded GPT-2 — everything Parts I, III and IV reference directly.
Qwen2_5_Architecture/	The full code-alongside-explanation treatment of RMSNorm, RoPE, GQA, SwiGLU that Part V condenses — read that document for the complete version of each.
Architecture_Deep_Dive/adi_*.py	This document's own build scripts — every formula and worked example above is real, runnable numpy code; rerun any part file directly to re-derive its numbers.
The_Architecture_of_Intelligence_2nd_Edition.pdf	The broader survey (27 concepts, 2017-2026) covering everything this document deliberately left out: data curation, serving infrastructure, agents, multimodality, evaluation. Read that one for the surrounding field; this one for the model itself.

Built from open, redistributable sources only: architecture code and configs are part of the Apache-2.0-licensed `transformers` Python package (Qwen2.5, DeepSeek-V3 modeling code and configs, all bundled locally) and openly published model cards / papers (DeepSeek-V2, DeepSeek-V3 technical reports). No model weights were downloaded to produce this document — only small config.json files (a few KB each) and code already installed on this machine.